COMPLETE COURSE

dbt Zero to Hero

Build production-grade analytics engineering workflows from scratch

Companion repo: shopstream-dbt/ · Adapter-agnostic · dbt Core 1.7+

WHAT'S INSIDE

Table of Contents

CHAPTER 01 · FOUNDATIONS

Analytics Engineering & the dbt Philosophy

Why dbt exists, what problem it solves, and why SQL-first transformations beat every alternative for analytical work.

The Transformation Problem

Every data team eventually hits the same wall. Raw data arrives from your source systems — Postgres, Salesforce, Stripe — and it lands in your warehouse in a format built for transactional databases, not analytics. Tables are normalized, column names are cryptic, monetary values are in cents, timestamps are in UTC without context, and foreign keys reference other tables that themselves need decoding.

Someone has to transform that raw mess into clean, trustworthy tables that analysts can actually use. For years, teams did this in one of three ways:

1. **ETL pipelines:** Python or Spark jobs that transform data before loading it. Expensive to write, painful to test, invisible to SQL analysts.
2. **BI tool logic:** Transformations buried inside Tableau calculated fields or Looker PDTs. Logic scattered across tools with no version control.
3. **The SQL file graveyard:** A shared drive full of files named `final_revenue_FINAL_v3_USE_THIS.sql`. No tests. No docs. No lineage.

dbt is the fourth way. It lets you write transformations in SQL — the language your whole team already knows — but brings software engineering practices to those SQL files: version control, testing, documentation, and a dependency graph that makes lineage explicit.

The Analytics Engineering Insight

Modern cloud warehouses (BigQuery, Snowflake, Redshift, DuckDB) are fast enough to transform data *inside* the warehouse rather than before loading it. This is the ELT (Extract, Load, Transform) pattern. dbt is purpose-built for the T in ELT.

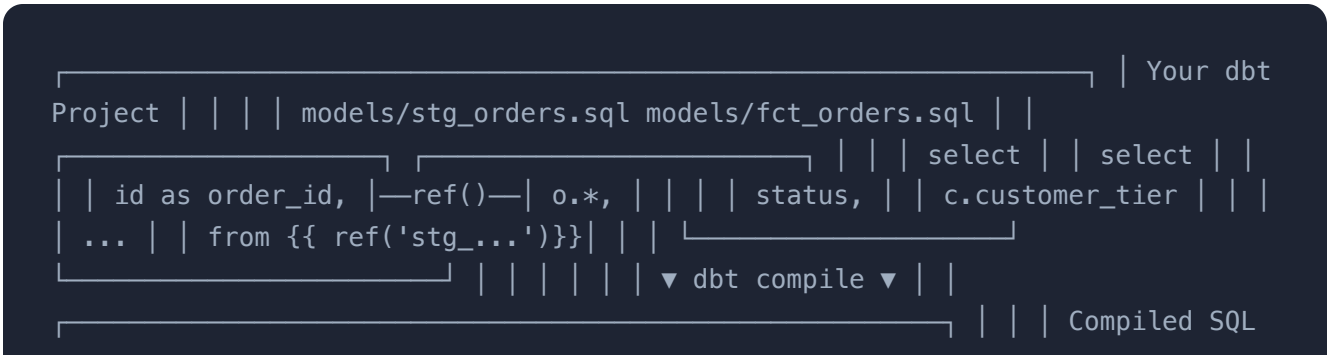
dbt doesn't move data. It doesn't connect to source systems. It takes data that's already in your warehouse and transforms it into better data, still in your warehouse.

What dbt Actually Is

At its core, dbt is a command-line tool that:

1. Takes your `.sql` files
2. Compiles them (resolves Jinja templates, `ref()` calls)
3. Runs the compiled SQL against your warehouse
4. Optionally runs tests against the results

That's it. dbt generates and runs SQL. Everything else — documentation, lineage, testing, packages — is built on top of that simple foundation.



```
(sent to warehouse) │ │ │ │ create view staging.stg_orders as (select ...) │ │
│ │ create table core.fct_orders as (select ...) │ │ │
└──────────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────┘
```

dbt Core vs dbt Cloud

Feature	dbt Core	dbt Cloud
Open source CLI	✅ Free	✅ Included
All adapters	✅	✅
Scheduler / jobs	❌ (use Airflow/Dagster)	✅ Built-in
Web IDE	❌	✅
Docs hosting	❌ (self-host)	✅
CI/CD integration	Manual setup	✅ Native
Cost	Free	Freemium / paid

This course uses dbt Core. Everything you learn here works identically in dbt Cloud — the Cloud simply wraps the Core CLI in a UI.

The Three Pains dbt Solves

Pain 1: No Lineage

With raw SQL files, you have no idea which query depends on which. Change a column name in one file and you won't know which downstream queries broke until they fail in production.

dbt's `ref()` function makes every dependency explicit and enforces execution order automatically.

Pain 2: No Testing

Without tests, data quality issues surface when a stakeholder finds a wrong number in a dashboard. With dbt, you can assert that `order_id` is always unique, that `status` only contains expected values, and that every order belongs to a real customer — and these assertions run automatically every time you build your models.

Pain 3: No Documentation

Without documentation, institutional knowledge lives in people's heads. When someone leaves, knowledge walks out the door. dbt lets you write documentation in the same files as your models, auto-generate a searchable doc site, and keep documentation permanently in sync with your code.

Mental Model: dbt Is Version Control for Your Transformations

Think of dbt the same way you think of Git for application code. Git didn't replace programmers or change what they write — it gave them a disciplined process for tracking changes, collaborating safely, and rolling back mistakes. dbt does the same for SQL: it doesn't change the SQL you write, it gives your SQL files the engineering discipline they've always deserved.

The Modern Data Stack Context

dbt occupies one specific role in the modern data stack. Understanding what it doesn't do is as important as understanding what it does:

Layer	Tool examples	Does dbt do this?
Data extraction	Fivetran, Airbyte, Stitch	❌ No
Data loading	Fivetran, custom loaders	❌ No
Data transformation	**dbt**	✅ Yes — this is dbt's job
Data storage	Snowflake, BigQuery, DuckDB	❌ No
Data consumption	Tableau, Looker, Metabase	❌ No

> **THE SHOPSTREAM SCENARIO**
>
> Throughout this course you'll work with **ShopStream**, a fictional e-commerce company. Their data is loaded from a PostgreSQL operational database into a warehouse via Fivetran. Your job is to transform it into clean, tested, documented tables that the BI team and data scientists can trust. The companion repo (`shopstream-dbt/`) contains the full working project.

CHAPTER 02 · FOUNDATIONS

Installation, Setup & Your First Project

Get dbt running, connect it to a warehouse, and build your first model in under 15 minutes.

Installing dbt Core

dbt Core is a Python package. Install the adapter for your warehouse:

bash

```
# PostgreSQL (used in this course)
pip install dbt-postgres

# Other adapters – install only one
pip install dbt-bigquery
pip install dbt-snowflake
pip install dbt-duckdb       # great for local development (no warel
pip install dbt-redshift
```

> **NO WAREHOUSE? USE DUCKDB**

> `pip install dbt-duckdb` gives you a fully functional warehouse running locally as a file. All examples in this course work with DuckDB. When you're ready to go to production, the only thing that changes is your `profiles.yml`.

Verify installation:

bash

```
dbt --version
# dbt Core: 1.7.x
# Registered adapter: postgres=1.7.x
```

Initializing a Project

bash

```
dbt init shopstream
cd shopstream
```

dbt creates this scaffold:

```
shopstream/ ├── dbt_project.yml ← project configuration ├── models/ │ ├──
example/ │ │ ├── my_first_dbt_model.sql │ │ └── my_second_dbt_model.sql │ └──
schema.yml ├── tests/ ├── macros/ ├── seeds/ ├── snapshots/ └── analyses/
```

Configuring profiles.yml

dbt separates project code (committed to git) from connection credentials (stored locally). Credentials live in `~/.dbt/profiles.yml` — never in your project.

~/.dbt/profiles.yml

```
shopstream:
  target: dev
  outputs:
    dev:
      type: postgres
      host: localhost
      user: myuser
      password: mypassword
      port: 5432
      dbname: shopstream
      schema: dbt_dev    # your personal dev schema
      threads: 4
```

Test the connection:

bash

```
dbt debug
# Connection test: [OK connection ok]
```

Your First Model

Delete the example models and create `models/stg_orders.sql`:

models/stg_orders.sql

```
-- Your first dbt model: clean up raw orders
select
    id          as order_id,
    customer_id,
    status,
    created_at  as order_placed_at
from raw.orders
```

Run it:

bash

```
dbt run
# Running with dbt=1.7.x
# Found 1 model
#
# Completed successfully
#
# Done. PASS=1 WARN=0 ERROR=0 SKIP=0 TOTAL=1
```

dbt created a **view** called `dbt_dev.stg_orders` in your warehouse. That's a dbt model — a SQL `SELECT` statement that dbt wraps in `CREATE VIEW AS` and executes for you.

Key CLI Commands

Command	What it does
`dbt run`	Build all models (create views/tables)
`dbt test`	Run all tests
`dbt build`	`run` + `test` together, in DAG order
`dbt compile`	Render Jinja but don't execute — writes to `target/compiled/`
`dbt seed`	Load CSV files from `seeds/` into the warehouse
`dbt snapshot`	Run snapshot models (SCD Type 2)
`dbt docs generate`	Generate the docs site artifacts
`dbt docs serve`	Serve the docs site at localhost:8080
`dbt deps`	Install packages from `packages.yml`
`dbt clean`	Delete `target/` and `dbt_packages/`

Node Selection

Every command accepts `--select` to target specific models:

bash

```bash
# Run a single model
dbt run --select stg_orders

# Run a model and all its downstream dependents (the + suffix)
dbt run --select stg_orders+

# Run all models in the staging/ directory
dbt run --select staging

# Run models with a specific tag
dbt run --select tag:daily

# Run models changed since the last run (Slim CI – covered in Ch. 1
dbt run --select state:modified+

# Exclude a model
dbt run --exclude stg_orders
```

Exercise 2.1 — First Run

Clone the companion repo and run the ShopStream project:

```
cd shopstream-dbt
dbt deps
dbt seed
dbt build
```

After it completes, query the warehouse directly: what tables/views were created? What schema are they in? What happens when you run `dbt build` a second time?

CHAPTER 03 · FOUNDATIONS

Project Structure & Configuration

How a production dbt project is organized and configured — from dbt_project.yml to environment-aware profiles.

dbt_project.yml Deep Dive

This is the heart of your project configuration. Every dbt project must have one:

`dbt_project.yml`

```yaml
name: 'shopstream'          # used in package references
version: '1.0.0'
config-version: 2

# Which profile in ~/.dbt/profiles.yml to use
profile: 'shopstream'

# Where dbt looks for files
model-paths:    ["models"]
analysis-paths: ["analyses"]
test-paths:     ["tests"]
seed-paths:     ["seeds"]
macro-paths:    ["macros"]
snapshot-paths: ["snapshots"]

# Build artifacts land here (never commit target/)
target-path: "target"
clean-targets: ["target", "dbt_packages"]

# Project-level variables — accessible via var() in any model
vars:
  shopstream:
    min_date: '2022-01-01'
    payment_methods: ['credit_card', 'paypal', 'bank_transfer', 'gi

# Model configuration — applied hierarchically by directory
models:
  shopstream:
    staging:
      +materialized: view       # all staging models are views
      +schema: staging          # created in the "staging" schema
    intermediate:
      +materialized: ephemeral  # no warehouse objects — become CTE
    marts:
      +materialized: table      # all mart models are tables
      core:
        +schema: core
      finance:
        +schema: finance
```

```
    marketing:
      +schema: marketing
```

> **THE + PREFIX**
>
> In `dbt_project.yml`, config keys that start with `+` are dbt model configs (materialization, schema, tags, etc.). Keys without `+` are directory names. This syntax prevents ambiguity when a directory name might conflict with a config key.

Directory Structure: The Three-Layer Convention

The most important structural decision in a dbt project is how you organize your `models/` directory. The industry-standard pattern uses three layers:

```
models/ ├── staging/ ← Layer 1: 1-to-1 with source tables | ├── _sources.yml ← source definitions | ├── _staging.yml ← model docs + tests | ├── stg_customers.sql | ├── stg_orders.sql | └── stg_payments.sql | ├── intermediate/ ← Layer 2: complex business logic (ephemeral) | ├── int_orders_with_payments.sql | └── int_customer_order_history.sql | └── marts/ ← Layer 3: clean tables for BI/ML ├── core/ | ├── _core.yml | ├── dim_customers.sql | └── fct_orders.sql └── finance/ └── fct_revenue_daily.sql
```

Each layer has strict rules about what it may and may not do:

Layer	May do	Must NOT do
Staging	Rename, cast, simple flag derivations	Join to other models, aggregate, apply complex logic
Intermediate	Complex joins, aggregations, business logic	Be referenced by BI tools directly (ephemeral = invisible)

Layer	May do	Must NOT do
Marts	Produce final, BI-ready dim/fact tables	Read directly from sources (must go through staging)

Schema Naming in Practice

In production you want clean schema names: `staging`, `core`, `finance`. In development, multiple engineers sharing a warehouse need isolation. dbt's default schema naming concatenates them: `dbt_alice_staging`. You can override this by implementing the `generate_schema_name` macro:

macros/generate_schema_name.sql

```
{% macro generate_schema_name(custom_schema_name, node) -%}
    {%- set default_schema = target.schema -%}

    {%- if target.name == 'prod' -%}
        {%- if custom_schema_name is none -%}
            {{ default_schema }}
        {%- else -%}
            {{ custom_schema_name | trim }}
        {%- endif -%}
    {%- else -%}
        {# In dev: all models land in the developer's personal sche
        {{ default_schema }}
    {%- endif -%}
{%- endmacro %}
```

With this macro:

- **prod target:** `stg_orders` → `staging.stg_orders`, `fct_orders` → `core.fct_orders`
- **dev target:** `stg_orders` → `dbt_alice.stg_orders`, `fct_orders` → `dbt_alice.fct_orders`

Environment Variables

Never hardcode credentials. Use `env_var()` in profiles and optionally in models:

~/.dbt/profiles.yml

```yaml
shopstream:
  target: dev
  outputs:
    dev:
      type: postgres
      host: localhost
      user: "{{ env_var('DBT_USER') }}"
      password: "{{ env_var('DBT_PASSWORD') }}"
      port: 5432
      dbname: shopstream
      schema: "{{ env_var('DBT_DEV_SCHEMA', 'dbt_dev') }}"  # defau
      threads: 4
```

bash

```bash
export DBT_USER=myuser
export DBT_PASSWORD=mysecret
dbt run
```

The target Object

Inside any model or macro, the `target` object gives you context about the current execution environment:

sql (in a model)

```
-- Example: add a debug column only in dev
select
    *
    {% if target.name != 'prod' %}
    , '{{ target.name }}' as _dbt_target   -- shows up in dev, stri
    {% endif %}
from my_source_table
```

Variable	Example value
`target.name`	`"dev"`, `"prod"`, `"ci"`
`target.schema`	`"dbt_alice"`
`target.database`	`"shopstream"`
`target.type`	`"postgres"`, `"bigquery"`
`target.threads`	`4`

CHAPTER 04 · CORE CONCEPTS

Models & Materializations

The four materializations every dbt engineer must master — and when to use each one.

What Is a Model?

A dbt model is a `.sql` file containing a single `SELECT` statement. That's it. dbt wraps that `SELECT` in a `CREATE VIEW AS` or `CREATE TABLE AS` statement and executes it against your warehouse. You never write `CREATE`, `DROP`, or `INSERT` in a model — dbt handles all of that.

models/stg_customers.sql

```
-- This is a complete, valid dbt model.
-- dbt will turn this into: CREATE VIEW staging.stg_customers AS (.
select
    id          as customer_id,
    lower(email) as email,
    is_active,
    created_at
from raw.customers
```

The Four Materializations

1. view (default)

dbt creates a **SQL view**. No data is stored — the query runs every time the view is queried. Views are always "fresh" but can be slow if the underlying tables are large.

Config options (pick one)

```
-- Option A: in-file config block (takes precedence over dbt_projec
{{ config(materialized='view') }}

select ...

-- Option B: dbt_project.yml (applies to all models in a directory)
models:
  shopstream:
    staging:
      +materialized: view
```

Use views for: Staging models, frequently-changed logic, models where freshness matters more than performance.

2. table

dbt creates a **physical table**. On each run, dbt drops the existing table and rebuilds it from scratch. Tables are fast to query but can take a long time to build if they scan large source data.

models/marts/core/dim_customers.sql

```
{{
  config(
    materialized = 'table',
    tags         = ['core', 'daily']
  )
}}

select
    c.customer_id,
    c.email,
    coalesce(oh.lifetime_value, 0) as lifetime_value
from {{ ref('stg_customers') }} c
left join {{ ref('int_customer_order_history') }} oh using (custome
```

Use tables for: Mart models queried by BI tools, models that are expensive to re-compute on every query, anything downstream of complex joins.

3. incremental

The most powerful materialization. On the first run, builds the full table. On subsequent runs, *only processes new or changed rows*, then appends or merges them into the existing table. This makes large-table builds extremely fast.

`models/marts/finance/fct_revenue_daily.sql`

```
{{
  config(
    materialized         = 'incremental',
    unique_key           = 'revenue_date',
    incremental_strategy = 'merge'
  )
}}

select
    order_date                                                  as reve
    count(*)                                                    as tota
    sum(case when is_completed then revenue else 0 end)         as gros
from {{ ref('fct_orders') }}

{% if is_incremental() %}
    -- On incremental runs: only process the last 3 days
    -- (covers late-arriving data)
    where order_date >= (
        select max(revenue_date) - interval '3 days'
        from {{ this }}
    )
{% endif %}

group by 1
```

> **THE IS_INCREMENTAL() FILTER IS CRITICAL**
>
> Without `{% if is_incremental() %}`, your model will scan the entire source table on every run, defeating the purpose of incremental materialization. The filter tells dbt which source rows to process on incremental runs. On a full refresh (`dbt run --full-refresh`), `is_incremental()` evaluates to `false` and the filter is skipped.

INCREMENTAL STRATEGIES

Strategy	Behavior	Requires unique_key?	Best for
`append`	INSERT new rows only, never update	No	Immutable event logs
`merge`	MERGE — upsert by unique_key	Yes	Fact tables with corrections
`delete+insert`	DELETE matching rows, then INSERT	Yes	Postgres/Redshift (no native MERGE)
`insert_overwrite`	Overwrite entire partitions	No	BigQuery/Spark partitioned tables

4. ephemeral

Ephemeral models are never materialized as objects in the warehouse. Instead, dbt inlines the model's SQL as a **CTE** inside any model that references it via `ref()`. They're invisible to BI tools but let you organize complex logic into named, reusable pieces.

`models/intermediate/int_orders_with_payments.sql`

```
-- No config block needed — configured as ephemeral in dbt_project.y
-- This entire model becomes a CTE in downstream models

with orders as (
    select * from {{ ref('stg_orders') }}
),

payments_agg as (
    select
        order_id,
        max(case when is_successful then amount end) as payment_amou
        max(case when is_successful then payment_method end) as payr
    from {{ ref('stg_payments') }}
    group by 1
)

select
    o.*,
    coalesce(p.payment_amount, 0) as order_amount,
    p.payment_method
from orders o
left join payments_agg p using (order_id)
```

When `fct_orders.sql` does `{{ ref('int_orders_with_payments') }}`, dbt inlines the above as a CTE. No separate database object is created.

Choosing the Right Materialization

- **Staging** → view (always fresh, small transformations)
- **Intermediate** → ephemeral (complex logic, not queried directly)
- **Dimension tables** → table (BI tools query these constantly)
- **Fact tables, small-medium** → table
- **Fact tables, large / append-only** → incremental

- **Daily/hourly aggregations** → incremental (only reprocess recent data)

Config Precedence

dbt resolves model configuration in this order (higher = wins):

1. **In-file `config()` block** — highest priority
2. `dbt_project.yml` — directory-level defaults
3. **dbt defaults** — `view` if nothing is specified

Advanced Config Options

sql

```sql
{{
  config(
    materialized        = 'table',
    schema              = 'core',           -- override project.yml
    alias               = 'orders',         -- table name (default:
    tags                = ['daily', 'pii'], -- for selection and go
    enabled             = true,             -- set false to skip a
    on_schema_change    = 'sync_all_columns', -- incremental: handl
    grants              = {'select': ['role_bi_reader']},  -- auto-
    post_hook           = "analyze {{ this }}"  -- run SQL after bu
  )
}}
```

Exercise 4 — Materializations

In the companion repo, open `exercises/ch04/exercise.md` and complete all four exercises. The key one: change `stg_customers` to a table, run it twice, then change it back to a view. Use your warehouse's query history to see the compile/execute difference between the two materializations.

CHAPTER 05 · CORE CONCEPTS

Sources, Seeds & Freshness

Declare your raw data's contract, load reference data, and alert when sources go stale.

Sources: Declaring Your Raw Data Contract

Without source definitions, your models refer to raw tables with bare SQL like `FROM raw.customers`. If that table moves, renames, or has a schema change, you won't know until a model fails. Sources solve this by declaring raw tables as first-class dbt objects with their own docs, tests, and freshness checks.

`models/staging/_sources.yml`

```yaml
version: 2

sources:
  - name: shopstream_raw           # the name used in source() calls
    description: "Raw data loaded from operational DB via Fivetran"
    database: shopstream           # warehouse database (optional,
    schema: raw                    # the actual warehouse schema na
    loader: fivetran
    loaded_at_field: _fivetran_synced  # column used for freshness

    freshness:
      warn_after:  {count: 12, period: hour}
      error_after: {count: 24, period: hour}

    tables:
      - name: customers
        description: "One row per registered customer account"
        columns:
          - name: id
            tests: [unique, not_null]
          - name: email
            tests: [unique, not_null]

      - name: orders
        freshness:
          warn_after: {count: 6, period: hour}  # override project-
        columns:
          - name: id
            tests: [unique, not_null]
          - name: status
            tests:
              - accepted_values:
                  values: ['placed','processing','completed','retur
```

Referencing Sources in Models

`models/staging/stg_customers.sql`

```
with source as (
    -- source() takes (source_name, table_name) — matches the yml ab
    select * from {{ source('shopstream_raw', 'customers') }}
),

renamed as (
    select
        id          as customer_id,
        lower(email) as email,
        is_active,
        created_at,
        updated_at
    from source
)

select * from renamed
```

The `source()` function does two things `FROM raw.customers` cannot: it registers a lineage edge in the DAG (visible in dbt docs), and it lets dbt run freshness checks against the source.

Source Freshness

Run freshness checks independently from your models — typically in a separate scheduled job that runs every hour:

bash

```
dbt source freshness
```

dbt queries `max(_fivetran_synced)` for each source table and compares it to the current time. If data is stale, it warns or errors according to your thresholds — before your models even run.

```
dbt source freshness output: shopstream_raw.customers ✅ last loaded 2h ago
(warn at 12h, error at 24h) shopstream_raw.orders ⚠️ last loaded 8h ago (warn
```

```
  at 6h — WARNING) shopstream_raw.payments ✅ last loaded 1h ago
```

Seeds: Loading Reference Data

Seeds are CSV files in your `seeds/` directory that dbt loads into the warehouse as tables. They're ideal for small, slowly-changing reference data that doesn't come from a source system:

- Country code → country name mappings
- Product category taxonomies
- Marketing channel definitions
- Test fixtures for development

`seeds/country_codes.csv`

```
code,name,region
US,United States,North America
UK,United Kingdom,Europe
CA,Canada,North America
AU,Australia,Asia Pacific
DE,Germany,Europe
SG,Singapore,Asia Pacific
```

`bash`

```
dbt seed                # loads all seeds
dbt seed --select country_codes   # loads one specific seed
```

Reference a seed in a model just like any other model using `ref()`:

`sql`

```
select
    o.order_id,
    o.shipping_country_code,
    cc.name       as country_name,
    cc.region
from {{ ref('fct_orders') }} o
left join {{ ref('country_codes') }} cc
    on o.shipping_country_code = cc.code
```

Configuring Seeds

dbt_project.yml

```
seeds:
  shopstream:
    +schema: seeds           # seeds land in a "seeds" schema
    raw_customers:
      +column_types:         # explicit type casting for CSV columns
        id: integer
        created_at: timestamp
        is_active: boolean
```

> **SEEDS ARE NOT FOR LARGE DATA**
>
> Seeds are loaded by reading the CSV file and generating `INSERT` statements. This works fine for up to a few thousand rows, but is not designed for large datasets. If your reference data has 100k+ rows, load it through your EL pipeline instead and declare it as a source.

Sources vs Seeds vs Models — When to Use Each

Type	Where data comes from	How to reference
Source	Loaded by your EL tool (Fivetran, etc.)	`source('name', 'table')`
Seed	CSV file in your `seeds/` directory	`ref('seed_name')`
Model	Built by dbt from sources/seeds	`ref('model_name')`

CHAPTER 06 · CORE CONCEPTS

Testing & Documentation

Make your data trustworthy. Every assertion you don't write is a bug your stakeholders will find first.

Why Testing Matters More Than You Think

The most dangerous data is data that looks correct but isn't. A `customer_id` that's accidentally duplicated, an `order_amount` that's negative because of a currency conversion bug, a `status` value that should be `'completed'` but a source system started writing `'COMPLETED'` — these are the issues that erode trust in your data platform. dbt tests catch them automatically, before they reach dashboards.

Generic Tests

dbt ships with four built-in generic tests that cover the most common data quality assertions:

`models/staging/_staging.yml`

```yaml
version: 2

models:
  - name: stg_orders
    columns:
      - name: order_id
        tests:
          - unique          # no duplicate order_ids
          - not_null        # order_id can never be null

      - name: status
        tests:
          - accepted_values:
              values: ['placed', 'processing', 'completed', 'return

      - name: customer_id
        tests:
          - not_null
          - relationships:   # every customer_id must exist in stg_
              to: ref('stg_customers')
              field: customer_id
```

Run them:

bash

```bash
dbt test --select stg_orders
# 4 tests run. PASS=4 WARN=0 ERROR=0
```

Under the hood, dbt compiles each test into a SQL query that returns rows only when the test *fails*. If zero rows are returned, the test passes. If any rows are returned, the test fails and dbt shows you the offending rows:

Compiled test SQL (generated by dbt)

```sql
-- unique test for stg_orders.order_id
select
    order_id,
    count(*) as n
from shopstream.staging.stg_orders
group by order_id
having count(*) > 1
```

Singular Tests

Singular tests are custom SQL assertions you write yourself in the `tests/` directory. They're for business logic too specific for generic tests. Like generic tests, they must return **zero rows to pass**.

`tests/assert_payments_match_order_items.sql`

```sql
-- Assert: for every completed order, the successful payment amount
-- must equal the sum of order item line totals (within 1 cent roun

with order_item_totals as (
    select order_id, sum(line_total_cents) as items_total_cents
    from {{ ref('stg_order_items') }}
    group by 1
),

payments as (
    select order_id, amount_cents as payment_cents
    from {{ ref('stg_payments') }}
    where is_successful
),

orders as (
    select order_id
    from {{ ref('stg_orders') }}
    where status = 'completed'
)

-- Return rows where there's a discrepancy greater than 1 cent
select
    o.order_id,
    oi.items_total_cents,
    p.payment_cents,
    abs(oi.items_total_cents - p.payment_cents) as discrepancy_cents

from orders o
join order_item_totals oi using (order_id)
join payments p using (order_id)

where abs(oi.items_total_cents - p.payment_cents) > 1
```

Custom Generic Tests

You can write your own reusable generic tests as macros. Any macro in `macros/` whose name starts with `test_` becomes a generic test:

macros/test_is_positive.sql

```
-- Custom generic test: assert all values in a column are positive
-- Usage in schema.yml:
--   tests:
--     - is_positive

{% test is_positive(model, column_name) %}

select *
from {{ model }}
where {{ column_name }} is not null
  and {{ column_name }} <= 0

{% endtest %}
```

Apply it in _staging.yml

```
- name: quantity
  tests:
    - not_null
    - is_positive     # ← your custom generic test
```

Test Configuration: Severity and Thresholds

Not all test failures are equally urgent. Configure severity per test:

_staging.yml

```yaml
- name: email
  tests:
    - unique:
        severity: error     # fail the run (default)
    - not_null:
        severity: warn      # emit a warning but don't fail

- name: revenue
  tests:
    - dbt_utils.expression_is_true:
        expression: ">= 0"
        severity: error
        error_if: ">10"     # only error if MORE than 10 rows fail
        warn_if:  ">0"      # warn if even 1 row fails
```

Documentation

Schema YAML Documentation

Document models and columns directly in your `.yml` files:

`models/marts/core/_core.yml`

```yaml
version: 2

models:
  - name: fct_orders
    description: >
      Order fact table. One row per order (grain = order_id).
      The primary table for revenue analysis. Revenue figures are i
    columns:
      - name: order_id
        description: Primary key. Surrogate from the source system.
        tests: [unique, not_null]
      - name: revenue
        description: >
          Order revenue in USD. Only populated for completed orders
          zero for cancelled or failed orders. Does NOT include retu
        tests:
          - dbt_utils.expression_is_true:
              expression: ">= 0"
```

Doc Blocks: Reusable Description Strings

For lengthy descriptions shared across multiple models, use doc blocks:

docs/metrics.md

```
{% docs revenue %}
Order revenue in USD. Calculated as the successful payment amount.
Only populated for orders with status = 'completed'.
Zero for cancelled or failed orders. Does NOT net out returns — use
`net_revenue` (gross minus refunds) for financial reporting.
{% enddocs %}
```

Reference it in _core.yml

```yaml
- name: revenue
  description: "{{ doc('revenue') }}"
```

Generating and Serving Docs

bash

```
dbt docs generate   # builds docs artifacts in target/
dbt docs serve      # opens the docs site at http://localhost:8080
```

The generated documentation includes:

- A searchable catalog of all models, sources, seeds, and tests
- An interactive DAG diagram showing all dependencies
- Column-level descriptions and test coverage
- Compiled SQL for every model

> **DOCUMENTATION IS A TEST FOR YOUR UNDERSTANDING**
>
> If you can't explain what a column means in plain English, you don't fully understand your data. Writing descriptions forces you to confront ambiguity and resolve it in the code. Teams that document their dbt models find bugs and missing business logic they never knew existed.

Exercise 6 — Tests & Docs

Open `exercises/ch06/exercise.md`. The key challenges: (1) write a singular test that asserts no customer has a negative lifetime value, (2) add a custom generic test `not_empty_string` and apply it to `stg_customers.email`, (3) run `dbt docs serve` and find the DAG for `fct_orders`.

CHAPTER 07 · CORE CONCEPTS

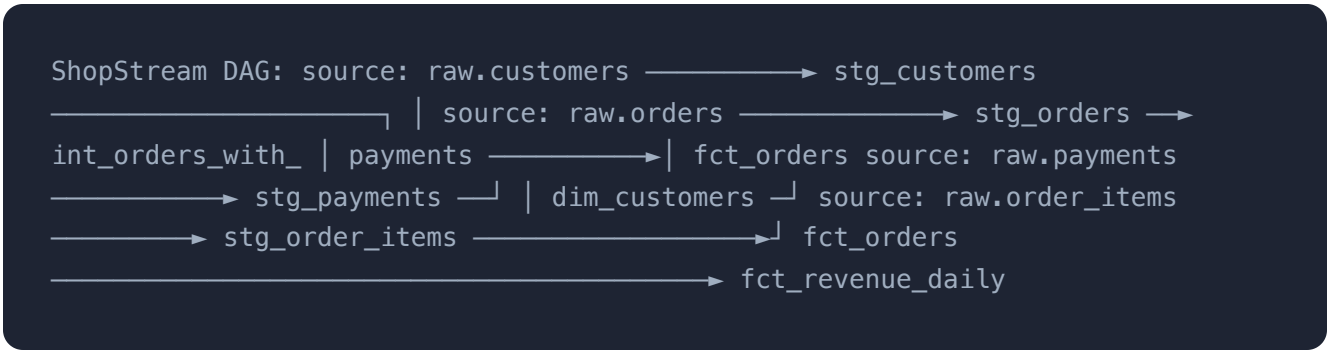
The DAG, ref() & Lineage

How dbt builds a dependency graph from your code and why ref() is the single most important function in dbt.

The DAG (Directed Acyclic Graph)

Every time you run dbt, it builds a DAG of your models — a graph where nodes are models and edges are dependencies (established by `ref()` calls). dbt uses this graph to:

- Determine execution order (parents always run before children)
- Run models in parallel where the graph allows
- Determine what's affected by a change (impact analysis)
- Generate the lineage visualization in docs



The ref() Function

`ref()` is how you reference another model. It's the mechanism that creates DAG edges and should be used *every time* you reference a model — never hardcode schema-qualified table names.

sql

```sql
-- ❌ Wrong – hardcodes schema, breaks in dev/prod, creates no DAG
select * from analytics.fct_orders

-- ✅ Correct – environment-aware, creates DAG edge, enables lineag
select * from {{ ref('fct_orders') }}
```

What `ref()` actually does at compile time:

1. Looks up the model's configured schema (respecting environment and generate_schema_name)
2. Replaces `{{ ref('fct_orders') }}` with the fully-qualified table name: `shopstream.core.fct_orders`
3. Registers a dependency edge in the DAG

Cross-Project ref (Advanced)

In multi-project dbt setups (dbt Mesh), you can ref models from other projects:

sql

```sql
-- Reference a model from another dbt project
select * from {{ ref('finance_dbt', 'fct_gl_entries') }}
```

Execution Order and Parallelism

dbt processes the DAG in topological order with configurable parallelism. Models at the same "level" with no dependencies between them run simultaneously:

```
Run order with threads=4: Round 1 (parallel): stg_customers, stg_orders,
stg_payments, stg_order_items (all sources → no deps between them) Round 2
(parallel): int_orders_with_payments, int_customer_order_history (both need
staging models — run after round 1) Round 3 (parallel): dim_customers,
fct_orders (need intermediate models) Round 4: fct_revenue_daily (needs
fct_orders)
```

Set `threads` in your profile to control max parallelism. More threads = faster builds, but more warehouse concurrency load.

Model Selection and the DAG

The `--select` syntax uses the DAG for powerful targeting:

bash

```bash
# Run only models upstream of fct_orders (its ancestors)
dbt run --select +fct_orders

# Run fct_orders and everything downstream of it
dbt run --select fct_orders+

# Run everything between stg_orders and fct_orders inclusive
dbt run --select stg_orders+fct_orders   # path through the DAG

# Run changed models and all their descendants (Slim CI pattern)
dbt run --select state:modified+

# Intersection: models tagged "daily" that are also downstream of stg_orders
dbt run --select tag:daily,+stg_orders
```

this: Referencing the Current Model

In incremental models, you need to reference the model's own current state to compute the filter. The `this` variable provides this:

sql

```
{% if is_incremental() %}
    where order_date >= (
        select max(order_date) - interval '3 days'
        from {{ this }}    -- refers to the currently-existing fct_revenue_da
    )
{% endif %}
```

CHAPTER 08 · INTERMEDIATE SKILLS

Jinja Templating & Macros

Unlock the full power of dbt by writing reusable, adapter-aware, dynamic SQL with Jinja and macros.

Jinja in dbt

dbt uses the Jinja2 templating engine to make SQL dynamic. Jinja is evaluated at *compile time*, before the SQL is sent to your warehouse. The warehouse never sees Jinja — it only sees the rendered SQL output.

There are three Jinja constructs used in dbt:

Syntax	Purpose	Example
`{{ ... }}`	Expressions — output a value	`{{ ref('stg_orders') }}`
`{% ... %}`	Statements — control flow, no output	`{% if is_incremental() %}`
`{# ... #}`	Comments — stripped at compile time	`{# TODO: add indexes #}`

Variables: var()

Variables defined in `dbt_project.yml` are accessible anywhere with `var()`:

dbt_project.yml

```
vars:
  shopstream:
    payment_methods: ['credit_card', 'paypal', 'bank_transfer', 'gi
    min_date: '2022-01-01'
```

sql usage in a model

```
-- Use var() to reference the list
select *
from {{ ref('stg_payments') }}
where payment_method in ({{ var('payment_methods') | join("','") |

-- Or in tests:
-- accepted_values:
--   values: "{{ var('payment_methods') }}"

-- Override a variable at runtime:
-- dbt run --vars '{"min_date": "2023-01-01"}'
```

Control Flow

sql

```
-- if/elif/else
{% if target.name == 'prod' %}
    select * from production_table
{% elif target.name == 'ci' %}
    select * from ci_table limit 10000
{% else %}
    select * from dev_table limit 1000
{% endif %}

-- for loop — generate multiple UNION ALL branches
{% set payment_methods = ['credit_card', 'paypal', 'bank_transfer'] %}

{% for method in payment_methods %}
    select
        '{{ method }}'  as payment_method,
        count(*)        as order_count
    from {{ ref('stg_payments') }}
    where payment_method = '{{ method }}'
    {% if not loop.last %} union all {% endif %}
{% endfor %}
```

Writing Macros

Macros are reusable Jinja functions. Any `.sql` file in the `macros/` directory is automatically available across your entire project.

Simple Utility Macro

`macros/cents_to_dollars.sql`

```
{% macro cents_to_dollars(column_name, precision=2) %}
    ({{ column_name }} / 100.0)::numeric(16, {{ precision }})
{% endmacro %}
```

`Usage in a model`

```
select
    order_id,
    amount_cents,
    {{ cents_to_dollars('amount_cents') }}          as amount,
    {{ cents_to_dollars('amount_cents', 4) }}       as amount_preci
from {{ ref('stg_payments') }}
```

Macro That Generates SQL

macros/safe_divide.sql

```
-- Returns null instead of erroring on division by zero
{% macro safe_divide(numerator, denominator) %}
    case
        when ({{ denominator }}) = 0 or ({{ denominator }}) is null
        then null
        else ({{ numerator }}) / nullif(({{ denominator }}), 0)
    end
{% endmacro %}
```

Usage

```
select
    customer_id,
    returned_orders,
    completed_orders,
    {{ safe_divide('returned_orders', 'completed_orders') }} as retu
from {{ ref('dim_customers') }}
```

Adapter-Aware Macros with dispatch

The most powerful macro pattern: write adapter-specific implementations that dbt selects automatically based on the warehouse you're connected to. This is how dbt-utils makes its macros work across every adapter.

macros/current_timestamp.sql

```
-- "Dispatcher" macro: calls the right implementation based on adap
{% macro current_timestamp() %}
    {{ return(adapter.dispatch('current_timestamp', 'shopstream')()
{% endmacro %}

-- PostgreSQL / Redshift implementation
{% macro postgres__current_timestamp() %}
    now()
{% endmacro %}

{% macro redshift__current_timestamp() %}
    getdate()
{% endmacro %}

-- BigQuery implementation
{% macro bigquery__current_timestamp() %}
    current_timestamp()
{% endmacro %}

-- Snowflake implementation
{% macro snowflake__current_timestamp() %}
    convert_timezone('UTC', current_timestamp())::timestamp_ntz
{% endmacro %}

-- DuckDB implementation
{% macro duckdb__current_timestamp() %}
    current_timestamp
{% endmacro %}
```

Usage (works on all adapters)

```
select
    order_id,
    {{ current_timestamp() }} as loaded_at
from {{ ref('stg_orders') }}
```

The generate_schema_name Macro (Revisited)

This is one of the most important macros to implement in any production project. It gives you control over how dbt names the schemas it writes to — critical for running dev and prod in the same warehouse without collision:

macros/generate_schema_name.sql

```
{% macro generate_schema_name(custom_schema_name, node) -%}
    {%- set default_schema = target.schema -%}

    {%- if target.name == 'prod' -%}
        -- In prod: use the custom schema name from config (staging
        {%- if custom_schema_name is none -%}
            {{ default_schema }}
        {%- else -%}
            {{ custom_schema_name | trim }}
        {%- endif -%}
    {%- else -%}
        -- In dev/ci: all models in the developer's personal schema
        {{ default_schema }}
    {%- endif -%}
{%- endmacro %}
```

Calling Macros from Other Macros

macros/log_run_info.sql

```
-- Macro that calls another macro and logs information
{% macro log_run_info(model_name) %}
    {% set msg = "Building " ~ model_name ~ " in schema: " ~ target
    {{ log(msg, info=True) }}
{% endmacro %}
```

run_query: Executing SQL in Macros

Macros can execute SQL and use the results at compile time. Use this carefully — it runs a query during compilation, not execution:

macros/get_column_values.sql

```
-- Get distinct values from a column to use in dynamic SQL
{% macro get_payment_methods() %}
    {% set query %}
        select distinct payment_method
        from {{ ref('stg_payments') }}
        order by 1
    {% endset %}

    {% set results = run_query(query) %}

    {% if execute %}
        {% set methods = results.columns[0].values() %}
        {{ return(methods) }}
    {% else %}
        {{ return([]) }}
    {% endif %}
{% endmacro %}
```

EXECUTE GUARD IS REQUIRED

Always wrap `run_query()` results in `{% if execute %}`. dbt processes the DAG in two passes: a "parsing" pass (where `execute = false`) and an "execution" pass. Without the guard, your macro will fail during parsing when the referenced table doesn't exist yet.

Exercise 8 — Macros

Complete `exercises/ch08/exercise.md`. Focus on exercise 8.3: write a macro that generates a dynamic pivot using a for loop over payment methods. The output should be one column per payment method showing total revenue.

CHAPTER 09 · INTERMEDIATE SKILLS

Snapshots & SCD Type 2

Track how your data changes over time with dbt snapshots — the production-grade implementation of slowly changing dimensions.

The Problem: Data Changes and You Lose History

Source systems are built to reflect the current state of the world, not history. When a customer changes their email, the old email is overwritten. When an order's status changes from `processing` to `completed`, you lose the record of when that transition happened. When a product's price changes, you can't answer "what was the price when this order was placed?"

This is the classic **Slowly Changing Dimension (SCD)** problem. dbt solves it with snapshots.

How dbt Snapshots Work

A dbt snapshot runs a query against a source table and compares the results to the previously-snapshotted version. For any rows that changed, it:

1. "Closes" the old record by setting `dbt_valid_to` to the current timestamp
2. Inserts a new record for the current state with `dbt_valid_from = now()` and `dbt_valid_to = null`

The result is a table where you can see the complete history of every record:

```
customer_snapshot (after 3 runs):

  id  email              country  dbt_valid_from      dbt_valid_to
  1   alice@example.com  US       2022-01-03 08:12    2023-06-01 12:00  ← closed
  1   alice@example.com  CA       2023-06-01 12:00    2023-11-01 09:00  ← closed
  1   alice@example.com  US       2023-11-01 09:00    null              ← CURREN
```

Writing a Snapshot

snapshots/customer_snapshot.sql

```
{% snapshot customer_snapshot %}

{{
    config(
      target_schema = 'snapshots',
      unique_key    = 'id',
      strategy      = 'timestamp',
      updated_at    = 'updated_at',  -- column that changes when a r
    )
}}

-- Snapshot the RAW source directly — not the staging model.
-- We want the unmodified original before any of our transformations
select * from {{ source('shopstream_raw', 'customers') }}

{% endsnapshot %}
```

bash

```
dbt snapshot
# Running with dbt=1.7.x
# Found 1 snapshot
# Snapshot shopstream.snapshots.customer_snapshot: [OK created]
```

The Two Snapshot Strategies

Strategy 1: timestamp

Uses a column that records when the row was last changed (e.g., `updated_at`). This is the most reliable strategy — it only detects a change when the source system explicitly marks the row as updated.

sql

```
{{
    config(
      strategy   = 'timestamp',
      updated_at = 'updated_at'   -- must be a datetime column
    )
}}
```

Requirement: Your source table must have a reliable `updated_at` column that the source system updates whenever a row changes. Many operational databases have this; some don't.

Strategy 2: check

Compares the current value of specific columns to the snapshotted value. If any of the watched columns differ, dbt treats the row as changed. Use this when your source table doesn't have an `updated_at` column.

sql

```
{{
    config(
      strategy   = 'check',
      check_cols = ['email', 'country', 'is_active']
      -- OR: check_cols = 'all'  to check every column
    )
}}
```

> **CHECK STRATEGY IS EXPENSIVE**
>
> With `check_cols = 'all'`, dbt must compare every column of every row on every run. On large tables this becomes very slow. Prefer the timestamp strategy whenever possible, and use check only for the specific columns you care about.

The Snapshot Metadata Columns

Column	Meaning
`dbt_scd_id`	Unique ID for this specific snapshot record (hash of PK + valid_from)
`dbt_updated_at`	When dbt last processed this record
`dbt_valid_from`	When this version of the record became active
`dbt_valid_to`	When this version was superseded. **NULL = current record**

Using Snapshots Downstream

Reference a snapshot exactly like a model using `ref()`:

`models/staging/stg_customer_history.sql`

```sql
with snapshot as (
    select * from {{ ref('customer_snapshot') }}
),

cleaned as (
    select
        id                                          as customer_id,
        lower(email)                                as email,
        upper(country)                              as country_code
        is_active,
        dbt_valid_from                              as valid_from,
        dbt_valid_to                                as valid_to,
        (dbt_valid_to is null)::boolean             as is_current,

        -- How long was this version active?
        coalesce(dbt_valid_to, current_timestamp)
            - dbt_valid_from                        as version_dura

    from snapshot
)

select * from cleaned
```

Point-in-Time Query

With snapshot history, you can answer "what was the state of this record at a specific point in time?" — essential for accurate historical reporting:

sql

```
-- What country was each customer in when they placed their order?
select
    o.order_id,
    o.order_placed_at,
    o.customer_id,
    c.country_code   as customer_country_at_order_time

from {{ ref('fct_orders') }} o
join {{ ref('stg_customer_history') }} c
    on  o.customer_id = c.customer_id
    and o.order_placed_at >= c.valid_from
    and o.order_placed_at <  coalesce(c.valid_to, '9999-12-31'::tim
```

SNAPSHOT BEST PRACTICES

- Always snapshot **raw sources**, not staging models. You want pre-transformation history.
- Run `dbt snapshot` on a **frequent schedule** (hourly or daily). Long gaps between runs mean changes in between are never captured.
- Keep snapshot queries **simple** — just `select * from source` with no complex logic. The logic belongs in downstream models.
- Set `target_schema = 'snapshots'` to keep snapshot tables in their own schema, separate from marts.

CHAPTER 10 · INTERMEDIATE SKILLS

Incremental Models

The most performance-critical materialization. Master it to build pipelines that process billions of rows in minutes.

Why Incremental?

A table materialization rebuilds the entire table every run. For a fact table with 500 million rows, that's 30+ minutes on every pipeline run — too slow for hourly or even daily schedules. Incremental models solve this by only processing *new or changed data* on each run and merging it into the existing table.

```
Full refresh (first run or --full-refresh flag):
┌──────────────────────────────────────────────────┐
│ Process ALL rows from source → write entire table │
└──────────────────────────────────────────────────┘

Incremental run (every subsequent run):
┌──────────────────────────────────────────────────┐
│ Process only NEW/CHANGED rows → merge into existing table│
└──────────────────────────────────────────────────┘
```

The Anatomy of an Incremental Model

`models/marts/finance/fct_revenue_daily.sql`

```
-- Step 1: declare the config
{{
  config(
    materialized         = 'incremental',
    unique_key           = 'revenue_date',        -- used for MERGE
    incremental_strategy = 'merge',
    on_schema_change     = 'sync_all_columns'
  )
}}

-- Step 2: write the full query
with orders as (

    select * from {{ ref('fct_orders') }}

    -- Step 3: filter to only new/changed data on incremental runs
    {% if is_incremental() %}
        where order_date >= (
            select max(revenue_date) - interval '3 days'
            from {{ this }}
        )
    {% endif %}

),

daily_agg as (

    select
        order_date                                                  as
        count(*)                                                    as
        sum(case when is_completed then revenue else 0 end)         as
        count(distinct customer_id)                                 as

    from orders
    group by 1

)

select * from daily_agg
```

All Four Incremental Strategies

append

Inserts new rows. Never updates or deletes. Safe for immutable event streams where each row is unique and never changes.

sql

```sql
{{
  config(
    materialized         = 'incremental',
    incremental_strategy = 'append'
    -- No unique_key needed — we're never deduplicating
  )
}}

select * from {{ ref('stg_payments') }}

{% if is_incremental() %}
    -- Only process payments created after the last run
    where paid_at > (select max(paid_at) from {{ this }})
{% endif %}
```

merge (default for most adapters)

UPSERT — matches new data to existing rows via `unique_key` and either updates existing rows or inserts new ones. Use this when source data can be corrected/updated after initial load.

sql

```
{{
  config(
    materialized         = 'incremental',
    unique_key           = 'order_id',
    incremental_strategy = 'merge'
    -- merge_update_columns = ['status', 'updated_at']  -- only upd
  )
}}

select * from {{ ref('stg_orders') }}

{% if is_incremental() %}
    where order_updated_at > (select max(order_updated_at) from {{
{% endif %}
```

delete+insert

First deletes all rows in the target table that match the incoming batch, then inserts the entire batch. Necessary for databases that don't support MERGE (Postgres, older Redshift).

sql

```
{{
  config(
    materialized         = 'incremental',
    unique_key           = ['order_date', 'country_code'],  -- compo
    incremental_strategy = 'delete+insert'
  )
}}
```

insert_overwrite

Replaces entire partitions in the target table. Available on BigQuery and Spark. The most efficient strategy for partitioned tables because it rewrites whole partition files atomically.

sql (BigQuery)

```
{{
  config(
    materialized           = 'incremental',
    incremental_strategy   = 'insert_overwrite',
    partition_by           = {
      "field": "order_date",
      "data_type": "date",
      "granularity": "day"
    }
  )
}}
```

Handling Schema Changes

What happens when you add a column to an incremental model? dbt's default behavior is to *ignore* the new column — existing runs just silently skip it. Configure `on_schema_change` to override this:

Value	Behavior
`'ignore'` (default)	New columns in the model are silently dropped during incremental runs
`'fail'`	Error immediately if schema differs — forces you to do a full refresh
`'append_new_columns'`	New columns are added with NULL for all existing rows
`'sync_all_columns'`	Add new columns AND remove deleted columns (recommended)

> **WHEN TO USE --FULL-REFRESH**
>
> You must run `dbt run --select my_model --full-refresh` when: (1) you change the `unique_key`, (2) you change the `incremental_strategy`, (3) you need to backfill

historical data with new logic, or (4) late-arriving data has corrupted a large portion of your table. Full refresh drops and rebuilds the table from scratch.

Incremental Predicates

On large partitioned tables, even the MERGE operation can scan the entire target table to find matching rows. `incremental_predicates` adds a filter on the *target* side of the merge to restrict the scan to recent partitions:

sql

```
{{
  config(
    materialized         = 'incremental',
    unique_key           = 'revenue_date',
    incremental_strategy = 'merge',
    incremental_predicates = [
      -- Only look at the last 7 days in the target table during mer
      -- Prevents a full table scan on the target side
      "dbt_internal_dest.revenue_date >= current_date - interval '7
    ]
  )
}}
```

DBT_INTERNAL_DEST AND DBT_INTERNAL_SOURCE

In incremental predicates, `dbt_internal_dest` refers to the existing target table and `dbt_internal_source` refers to the new data being merged in. These are aliases dbt creates in the compiled MERGE statement.

Microbatch Incremental (dbt 1.9+)

A newer incremental strategy designed for event-stream data. It automatically processes data in configurable time windows, retrying failed batches independently:

sql

```sql
{{
  config(
    materialized     = 'incremental',
    incremental_strategy = 'microbatch',
    event_time       = 'order_placed_at',    -- the timestamp colum
    begin            = '2022-01-01',         -- start of history
    batch_size       = 'day',               -- process one day at a
    lookback         = 3                     -- reprocess last 3 ba
  )
}}

select * from {{ ref('stg_orders') }}
-- No manual is_incremental() filter needed — dbt handles it
```

Exercise 10 — Incremental Models

Complete `exercises/ch10/exercise.md`. The most instructive exercise: run `fct_revenue_daily` with `--full-refresh`, note the rows processed in your warehouse query logs, then run it again without `--full-refresh` and compare. The difference is why incremental models exist.

CHAPTER 11 · INTERMEDIATE SKILLS

Packages & dbt-utils

Stand on the shoulders of giants. The dbt package ecosystem gives you hundreds of production-tested macros and tests.

The dbt Package Ecosystem

Packages are collections of models, macros, and tests that you can install into your project. They're published to the dbt Hub and installed via `packages.yml`.

packages.yml

```yaml
packages:
  - package: dbt-labs/dbt_utils
    version: [">=1.0.0", "<2.0.0"]

  - package: calogica/dbt_expectations
    version: [">=0.10.0", "<1.0.0"]

  - package: dbt-labs/codegen
    version: [">=0.12.0", "<1.0.0"]
```

bash

```bash
dbt deps    # downloads packages to dbt_packages/ (gitignored)
```

dbt-utils: The Essential Package

dbt-utils from dbt Labs is the standard library of the dbt ecosystem. You'll use it in almost every production project.

expression_is_true

The most flexible generic test — assert any SQL expression evaluates to true:

_staging.yml

```
- name: unit_price_cents
  tests:
    - dbt_utils.expression_is_true:
        expression: "> 0"

- name: quantity
  tests:
    - dbt_utils.expression_is_true:
        expression: ">= 1 and quantity <= 100"

# Cross-column expression test (model-level, not column-level)
models:
  - name: stg_order_items
    tests:
      - dbt_utils.expression_is_true:
          expression: "line_total_cents = quantity * unit_price_cen
```

unique_combination_of_columns

Test that a combination of columns is unique — essential for composite-key fact tables:

_finance.yml

```
models:
  - name: fct_revenue_daily
    tests:
      - dbt_utils.unique_combination_of_columns:
          combination_of_columns: ['revenue_date', 'shipping_country
```

date_spine

Generates a contiguous series of dates — essential for building time-series reports that need a row for every date even when no events occurred:

models/marts/core/dim_dates.sql

```
{{
  config(materialized='table')
}}

with date_spine as (

    {{ dbt_utils.date_spine(
        datepart   = "day",
        start_date = "cast('2022-01-01' as date)",
        end_date   = "current_date + interval '1 year'"
    ) }}

),

dates as (

    select
        date_day                                        as date_id,
        date_day,
        date_part('year', date_day)::integer            as year,
        date_part('month', date_day)::integer           as month_nu
        to_char(date_day, 'Month')                      as month_na
        date_part('quarter', date_day)::integer         as quarter,
        date_part('week', date_day)::integer            as week_of_
        date_part('dow', date_day)::integer             as day_of_w
        to_char(date_day, 'Day')                        as day_name
        (date_part('dow', date_day) in (0, 6))          as is_weeke
        date_trunc('week', date_day)::date              as week_sta
        date_trunc('month', date_day)::date             as month_st
        date_trunc('quarter', date_day)::date           as quarter_

    from date_spine

)

select * from dates
```

get_column_values

Query distinct values from a column at compile time — useful for dynamic pivots:

sql

```
{% set payment_methods = dbt_utils.get_column_values(
    table = ref('stg_payments'),
    column = 'payment_method'
) %}

select
    order_month,
    {% for method in payment_methods %}
        sum(case when payment_method = '{{ method }}'
                 then amount else 0 end)   as {{ method }}_revenue
        {% if not loop.last %},{% endif %}
    {% endfor %}

from {{ ref('stg_payments') }}
group by 1
```

pivot

Generates a pivot table without writing all the CASE WHEN manually:

sql

```
select
    order_month,
    {{ dbt_utils.pivot(
        column     = 'payment_method',
        values     = ['credit_card', 'paypal', 'bank_transfer', 'gi
        agg        = 'sum',
        then_value = 'amount'
    ) }}
from {{ ref('stg_payments') }}
group by 1
```

dbt-expectations: Great Expectations for dbt

dbt-expectations ports the Great Expectations test library to dbt, providing a comprehensive set of statistical and pattern-based tests:

_staging.yml

```yaml
- name: email
  tests:
    - dbt_expectations.expect_column_values_to_match_regex:
        regex: "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$"

- name: unit_price_cents
  tests:
    - dbt_expectations.expect_column_values_to_be_between:
        min_value: 100      # at least $1
        max_value: 1000000  # at most $10,000

- name: order_id
  tests:
    - dbt_expectations.expect_column_values_to_be_of_type:
        column_type: integer

models:
  - name: fct_orders
    tests:
      # Row count should be within 20% of yesterday's count
      - dbt_expectations.expect_table_row_count_to_be_between:
          min_value: 1
          max_value: 10000000
      # No column should be more than 5% null
      - dbt_expectations.expect_column_proportion_of_unique_values_
          column_name: customer_id
          min_value: 0.8
```

codegen: Generate Boilerplate Automatically

codegen saves hours of typing by generating staging model and schema YAML boilerplate from your source tables:

bash

```bash
# Generate a staging model SQL for a source table
dbt run-operation codegen.generate_base_model \
  --args '{"source_name": "shopstream_raw", "table_name": "customers

# Generate a sources.yml block for an entire schema
dbt run-operation codegen.generate_source \
  --args '{"schema_name": "raw", "generate_columns": true}'

# Generate a schema.yml for an existing model
dbt run-operation codegen.generate_model_yaml \
  --args '{"model_names": ["dim_customers", "fct_orders"]}'
```

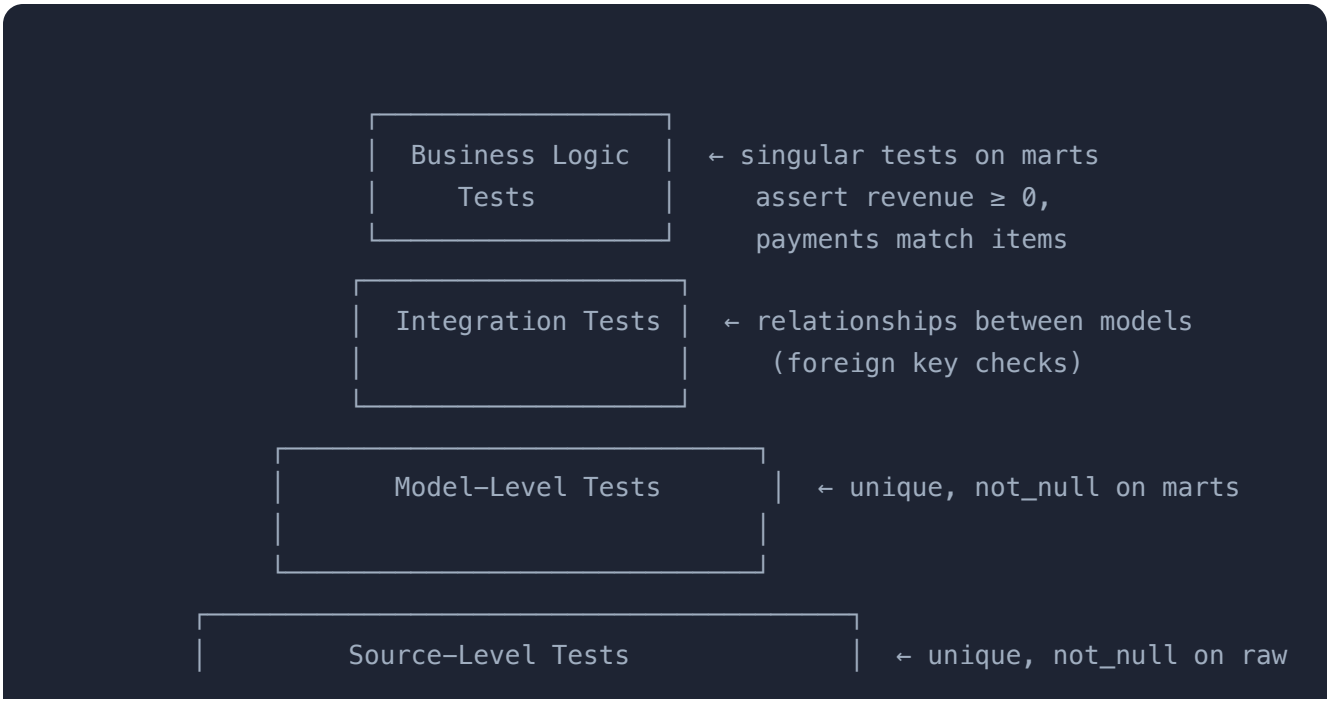
CHAPTER 12 · ADVANCED PATTERNS

Advanced Testing & Data Quality

Go beyond not_null and unique to build a comprehensive data quality framework that catches real production bugs.

The Testing Pyramid

Like software engineering's test pyramid, data testing has levels. Most teams over-invest in source tests and under-invest in business-logic tests:



```
                                                    accepted_values
```

Testing Strategies for Each Layer

Staging Layer Tests

Every staging model should have at minimum: `unique` + `not_null` on the primary key, `not_null` on all FK columns, and `accepted_values` on any enum columns:

_staging.yml

```yaml
- name: stg_payments
  columns:
    - name: payment_id
      tests: [unique, not_null]
    - name: order_id
      tests:
        - not_null
        - relationships:
            to: ref('stg_orders')
            field: order_id
    - name: payment_method
      tests:
        - accepted_values:
            values: "{{ var('payment_methods') }}"
    - name: amount_cents
      tests:
        - not_null
        - dbt_utils.expression_is_true:
            expression: "> 0"
```

Mart Layer Tests

Mart tests enforce business rules, not just technical constraints. They should test things that would break dashboards if violated:

_core.yml

```yaml
models:
  - name: fct_orders
    tests:
      # Grain test: every order appears exactly once
      - dbt_utils.unique_combination_of_columns:
          combination_of_columns: ['order_id']

      # Revenue must be non-negative
      - dbt_utils.expression_is_true:
          expression: "revenue >= 0"

      # Completed orders must have a payment
      - dbt_utils.expression_is_true:
          expression: "not (is_completed and not has_successful_payr

  - name: dim_customers
    tests:
      # LTV must be non-negative
      - dbt_utils.expression_is_true:
          expression: "lifetime_value >= 0"
      # Tier must be consistent with LTV
      - dbt_utils.expression_is_true:
          expression: |
            not (customer_tier = 'vip' and lifetime_value < 200)
```

Writing Powerful Singular Tests

Row Count Anomaly Detection

tests/assert_daily_order_volume_is_reasonable.sql

```
-- Fail if today's order count is more than 3x or less than 1/3 of
-- Catches sudden data pipeline failures or double-loads.

with daily_counts as (
    select
        order_date,
        count(*) as daily_orders
    from {{ ref('fct_orders') }}
    where order_date >= current_date - interval '8 days'
    group by 1
),

stats as (
    select
        avg(case when order_date < current_date then daily_orders e
        max(case when order_date = current_date - 1 then daily_orde
    from daily_counts
)

-- Return a row (trigger failure) if today's count is anomalous
select
    d.order_date,
    d.daily_orders,
    s.avg_7day,
    d.daily_orders / nullif(s.avg_7day, 0) as ratio

from daily_counts d
cross join stats s

where d.order_date = current_date
  and (
      d.daily_orders > s.avg_7day * 3    -- suspiciously high
      or d.daily_orders < s.avg_7day / 3  -- suspiciously low
  )
```

Referential Integrity Across Marts

`tests/assert_all_orders_have_items.sql`

```
-- Every completed order in fct_orders must have at least one order
-- A completed order with no items indicates a pipeline failure.

select
    o.order_id,
    o.status,
    o.order_date

from {{ ref('fct_orders') }} o
left join {{ ref('stg_order_items') }} oi using (order_id)

where o.is_completed
  and oi.order_item_id is null
```

Test Selection and CI Strategy

Not all tests are equal in urgency or runtime cost. Tag them and run selectively:

_core.yml

```
- name: order_id
  tests:
    - unique:
        tags: ['critical']    # run on every PR
    - not_null:
        tags: ['critical']

- name: revenue
  tests:
    - dbt_utils.expression_is_true:
        expression: ">= 0"
        tags: ['finance', 'daily']  # run only in daily pipeline
```

bash

```
# CI: run only critical tests (fast)
dbt test --select tag:critical

# Daily pipeline: run all tests
dbt test

# After a specific model change:
dbt test --select fct_orders+
```

store_failures: Debugging Test Failures

When a test fails, you want to see the actual offending rows. Configure `store_failures = true` to have dbt write the failing rows to a table in your warehouse:

dbt_project.yml

```
tests:
  shopstream:
    +store_failures: true          # store all test failures as table
    +schema: test_failures         # in this schema

    # Or per-model:
    marts:
      +store_failures: false       # don't store mart test failures
```

bash

```
dbt test --store-failures    # override at runtime
```

After a failure, query the failure table directly:

sql

```sql
-- See the actual rows that failed the unique test
select * from test_failures.unique_fct_orders_order_id
limit 100;
```

CHAPTER 13 · ADVANCED PATTERNS

Project Architecture at Scale

How to structure a dbt project that 10 engineers can work in simultaneously without stepping on each other.

The Fundamental Problem at Scale

A single-person dbt project can have a flat structure. A 10-person team working across multiple business domains can't. Without deliberate architecture, you end up with:

- Models in the wrong layer (analytics logic in staging, raw source queries in marts)
- Multiple models doing the same thing with slightly different logic
- No clear ownership — who's responsible when `fct_orders` breaks?
- All models sharing one giant schema, making permission management impossible

The Medallion Architecture in dbt

The industry-standard three-layer pattern maps cleanly to warehouse schemas and team responsibilities:

Naming Conventions

Consistent naming is how every engineer immediately understands what a model is and where it belongs:

Prefix	Layer	Example
`stg_`	Staging	`stg_customers`, `stg_orders`
`int_`	Intermediate	`int_orders_with_payments`
`dim_`	Mart — dimension	`dim_customers`, `dim_products`
`fct_`	Mart — fact	`fct_orders`, `fct_revenue_daily`
`rpt_`	Mart — report (optional)	`rpt_executive_summary`
`base_`	Base (pre-staging dedupe)	`base_customers`

Model Grain: The Single Most Important Design Decision

Every model must have a clearly-defined grain — the level of detail that each row represents. Document this explicitly:

`_core.yml`

```
- name: fct_orders
  description: >
    GRAIN: one row per order (order_id).
    Revenue figures in USD. Includes all orders regardless of statu

- name: fct_revenue_daily
  description: >
    GRAIN: one row per calendar day (revenue_date).
    Aggregated from fct_orders. Covers completed orders only.

- name: dim_customers
  description: >
    GRAIN: one row per customer (customer_id).
    Current snapshot of customer attributes plus lifetime metrics.
```

The Base Layer Pattern

For sources with duplicates that need deduplication before staging, add a `base_` layer between source and staging:

`models/staging/base/base_customers.sql`

```
-- Deduplicate the raw source before staging
with source as (
    select * from {{ source('shopstream_raw', 'customers') }}
),

deduped as (
    select *,
        row_number() over (
            partition by id
            order by updated_at desc
        ) as row_num

    from source
)

select * from deduped where row_num = 1
```

models/staging/stg_customers.sql

```
-- stg_customers reads from base_, not directly from source
with source as (
    select * from {{ ref('base_customers') }}
),
...
```

Cross-Domain References

Marts from different domains often need data from each other. Establish clear rules:

- ✅ **Marts can reference core:** `fct_revenue_daily` can reference `dim_customers`
- ✅ **Marts can reference staging:** when the staging model is the cleanest version of the data
- ❌ **Staging must NEVER reference marts:** creates circular dependencies
- ⚠️ **Finance should not reference Marketing's marts** (and vice versa): keep domains independent to avoid blast radius

Contract Models (dbt 1.5+)

For models that are consumed by external systems (BI tools, ML features, data products), you can enforce a strict schema contract. If the model's output doesn't match the declared schema, the run fails:

_core.yml

```yaml
- name: fct_orders
  config:
    contract:
      enforced: true    # fails if column types don't match declara
  columns:
    - name: order_id
      data_type: integer
      constraints:
        - type: not_null
        - type: primary_key   # enforced in the warehouse (if suppo
    - name: revenue
      data_type: numeric
      constraints:
        - type: not_null
        - type: check
          expression: "revenue >= 0"
```

Tags for Organization and Operations

Tags let you group models for selective execution, CI strategies, and monitoring:

dbt_project.yml

```
models:
  shopstream:
    marts:
      core:
        +tags: ['daily', 'core']
      finance:
        +tags: ['daily', 'finance', 'pii']
```

bash

```
# Run only models in the daily pipeline
dbt run --select tag:daily

# Run finance models but skip PII models (e.g. in a non-secure envi
dbt run --select tag:finance --exclude tag:pii
```

CHAPTER 14 · ADVANCED PATTERNS

Hooks, Grants & Operations

Run arbitrary SQL before or after your models build, automate permissions, and execute warehouse-level operations.

Hooks: SQL That Runs Around Your Models

Hooks let you execute arbitrary SQL before or after a model runs. They're how you handle things dbt doesn't natively support: granting permissions, analyzing tables for query planning, logging build metadata.

pre-hook and post-hook

In-file config

```
{{
  config(
    materialized = 'table',
    pre_hook     = "truncate table {{ this }}",
    post_hook    = [
      "analyze {{ this }}",
      "grant select on {{ this }} to role bi_reader"
    ]
  )
}}
```

dbt_project.yml (applies to all models in the directory)

```yaml
models:
  shopstream:
    marts:
      +post_hook:
        - "grant select on {{ this }} to role bi_reader"
        - "grant select on {{ this }} to role data_analyst"
```

on-run-start and on-run-end

These hooks run once per `dbt run` invocation, not per model. Use them for logging, warehouse setup, or cleanup:

dbt_project.yml

```yaml
on-run-start:
  - "create schema if not exists {{ target.schema }}"
  - "{{ log('Starting dbt run in ' ~ target.name, info=True) }}"

on-run-end:
  - "{{ log('Run complete. Models built: ' ~ results | length, info=
  # Grant on all new tables created during this run
  - "{% for res in results %}
      {% if res.status == 'success' %}
        grant select on {{ res.node.relation_name }} to role bi_re
      {% endif %}
    {% endfor %}"
```

Automating Grants with the grants Config

dbt 1.2+ supports a first-class `grants` config that's cleaner than post-hooks for permission management. dbt computes the diff between current and desired grants and only runs the necessary GRANT/REVOKE statements:

_core.yml

```yaml
models:
  - name: fct_orders
    config:
      grants:
        select: ['role_bi_reader', 'role_data_analyst']

  - name: dim_customers
    config:
      grants:
        select: ['role_bi_reader']
        # Note: dim_customers may have PII, so fewer roles get acce
```

dbt_project.yml (apply to all mart models)

```yaml
models:
  shopstream:
    marts:
      +grants:
        select: ['role_bi_reader']
```

Operations: Run SQL Without a Model

Operations (run via `dbt run-operation`) let you execute macros directly without building any model. Use them for one-time migrations, warehouse maintenance, or admin tasks:

macros/create_warehouse_objects.sql

```
{% macro create_schemas() %}
    {% set schemas = ['staging', 'core', 'finance', 'marketing', 'sa
    {% for schema in schemas %}
        {% set sql %}
            create schema if not exists {{ schema }};
            grant usage on schema {{ schema }} to role bi_reader;
        {% endset %}
        {% do run_query(sql) %}
        {{ log("Created schema: " ~ schema, info=True) }}
    {% endfor %}
{% endmacro %}
```

bash

```
dbt run-operation create_schemas
```

Common Operation Use Cases

macros/drop_old_dev_schemas.sql

```
-- Drop all dev schemas older than 30 days (Snowflake example)
{% macro drop_old_dev_schemas(days_old=30) %}

    {% set query %}
        select schema_name
        from information_schema.schemata
        where schema_name like 'dbt\_%' escape '\\'
          and created < current_date - {{ days_old }}
    {% endset %}

    {% set results = run_query(query) %}

    {% if execute %}
        {% for row in results %}
            {% set drop_sql = "drop schema if exists " ~ row[0] ~ "
            {{ log("Dropping: " ~ row[0], info=True) }}
            {% do run_query(drop_sql) %}
        {% endfor %}
    {% endif %}

{% endmacro %}
```

Hooks for Incremental Models: The Vacuum Pattern

After large incremental updates on Postgres/Redshift, tables can become bloated with dead rows. Run `VACUUM` automatically in a post-hook:

sql

```
{{
  config(
    materialized     = 'incremental',
    unique_key       = 'order_id',
    post_hook        = [
      "{% if target.type == 'postgres' %}vacuum analyze {{ this }}{%
    ]
  )
}}
```

CHAPTER 15 · ADVANCED PATTERNS

Exposures & the Semantic Layer

Document who uses your data and define business metrics once so they're consistent everywhere.

Exposures: Documenting Your Data Consumers

Exposures document the downstream consumers of your dbt models — dashboards, ML models, reverse ETL jobs. They appear in your DAG and let you answer: "If I change `fct_orders`, what BI reports will break?"

`models/marts/core/_core.yml (add exposures: block)`

```yaml
exposures:
  - name: revenue_dashboard
    type: dashboard
    maturity: high
    url: https://tableau.shopstream.com/views/Revenue/Overview
    description: >
      Executive revenue dashboard. Updated daily. Used by the CFO a
      Breaking this exposure will affect the weekly board report.

    depends_on:
      - ref('fct_orders')
      - ref('dim_customers')
      - ref('fct_revenue_daily')

    owner:
      name: Finance Analytics Team
      email: finance-analytics@shopstream.com

  - name: customer_ml_features
    type: ml
    maturity: medium
    description: >
      Feature table for the customer churn prediction model. Retrai

    depends_on:
      - ref('dim_customers')
      - ref('fct_orders')

    owner:
      name: ML Platform Team
      email: ml@shopstream.com
```

After running `dbt docs generate`, exposures appear as terminal nodes in the DAG with their own documentation pages. Running `dbt build --select +exposure:revenue_dashboard` builds every model that feeds the dashboard.

bash

```
# Build everything that feeds the revenue dashboard
dbt build --select +exposure:revenue_dashboard

# Test only models that feed a specific exposure
dbt test --select +exposure:revenue_dashboard
```

The Semantic Layer

The semantic layer solves the "metric sprawl" problem: when every team defines "revenue" slightly differently in their own BI tool. dbt's Semantic Layer lets you define metrics once in your dbt project and expose them consistently to every downstream tool.

> **METRICFLOW AND DBT SEMANTIC LAYER**
>
> dbt's Semantic Layer is powered by MetricFlow (open source). It requires dbt 1.6+ and is most fully featured in dbt Cloud. The concepts — semantic models and metrics — are part of dbt Core and can be compiled locally. Serving metrics to BI tools currently requires dbt Cloud's Semantic Layer API.

Semantic Models

A semantic model declares the business-level structure of a mart model — its entities, dimensions, and measures:

`models/marts/core/_semantic_models.yml`

```yaml
semantic_models:
  - name: orders
    description: The orders semantic model. Grain = one row per orde
    model: ref('fct_orders')

    entities:
      - name: order
        type: primary
        expr: order_id
      - name: customer
        type: foreign
        expr: customer_id

    dimensions:
      - name: order_date
        type: time
        type_params:
          time_granularity: day
      - name: status
        type: categorical
      - name: payment_method
        type: categorical
      - name: shipping_country_code
        type: categorical

    measures:
      - name: order_count
        agg: count
        expr: order_id
      - name: revenue
        agg: sum
        expr: revenue
        description: Sum of revenue from completed orders
      - name: avg_order_value
        agg: average
        expr: revenue
```

Metrics

Metrics are defined on top of semantic models. They're the single source of truth for KPIs:

models/marts/core/_metrics.yml

```yaml
metrics:
  - name: revenue
    description: Total revenue from completed orders (USD)
    type: simple
    type_params:
      measure: revenue

  - name: order_count
    description: Number of orders placed
    type: simple
    type_params:
      measure: order_count

  - name: avg_order_value
    description: Average revenue per completed order
    type: simple
    type_params:
      measure: avg_order_value

  # Derived metric: combines two metrics
  - name: revenue_per_customer
    description: Revenue divided by unique customer count
    type: derived
    type_params:
      expr: revenue / customer_count
      metrics:
        - name: revenue
        - name: customer_count

  # Ratio metric
  - name: return_rate
    description: Proportion of orders that were returned
    type: ratio
    type_params:
      numerator: returned_orders
      denominator: order_count
```

Querying Metrics with MetricFlow

bash

```bash
# Query a metric from the CLI
mf query --metrics revenue --group-by order_date__month

# Multiple metrics, filtered
mf query \
  --metrics revenue,order_count,avg_order_value \
  --group-by order_date__month,shipping_country_code \
  --where "order_date >= '2022-01-01'"

# Validate your semantic models are correctly defined
mf validate-configs
```

CHAPTER 16 · PRODUCTION

CI/CD & Slim CI

The production deployment patterns that let you ship changes to your data models safely and quickly.

Why CI/CD for dbt?

Without CI/CD, dbt changes go through an informal process: an engineer makes a change, runs it in dev, and merges directly to main. The problems:

- Bugs reach production before any tests run
- No record of what changed and when
- Large batched changes make rollbacks painful
- Every full run takes 20+ minutes, making CI impractical

Slim CI addresses all of these. It runs tests *only on changed models*, making CI runs fast enough to be practical on every PR.

The State Concept

dbt's state system compares the current project to a previously-built "production" state (captured in `manifest.json`). This lets you identify exactly which models changed:

bash

```bash
# Save the production manifest after a successful prod run
cp target/manifest.json prod_manifest.json

# On a PR branch: run only changed models and their downstream depe
dbt run \
  --select state:modified+ \
  --defer \
  --state ./prod_manifest.json
```

Key flags:

- `state:modified+` — select models whose definition changed, plus all their descendants
- `--defer` — for models NOT being rebuilt, use the production version instead of failing
- `--state ./prod_manifest.json` — the manifest to compare against

GitHub Actions CI Workflow

`.github/workflows/ci.yml`

```yaml
name: dbt CI

on:
  pull_request:
    branches: [main]

jobs:
  dbt-ci:
    runs-on: ubuntu-latest
    environment: ci

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dbt
        run: pip install dbt-postgres

      - name: Install dbt packages
        run: dbt deps

      - name: Download production manifest
        # Pull the latest prod manifest from S3/GCS/artifact storag
        run: |
          aws s3 cp s3://shopstream-dbt-artifacts/manifest.json \
            ./prod_manifest.json
        env:
          AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
          AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }

      - name: Run dbt Slim CI
        run: |
          dbt build \
            --select state:modified+ \
            --defer \
```

```
            --state ./prod_manifest.json \
            --target ci
        env:
          DBT_USER: ${{ secrets.DBT_CI_USER }}
          DBT_PASSWORD: ${{ secrets.DBT_CI_PASSWORD }}
          DBT_HOST: ${{ secrets.DBT_CI_HOST }}

      - name: Upload manifest artifact
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: dbt-manifest
          path: target/manifest.json
```

Production Deployment Workflow

`.github/workflows/deploy.yml`

```yaml
name: dbt Production Deploy

on:
  push:
    branches: [main]

jobs:
  dbt-deploy:
    runs-on: ubuntu-latest
    environment: production

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }

      - name: Install dbt
        run: pip install dbt-postgres

      - name: dbt deps
        run: dbt deps

      - name: Check source freshness
        run: dbt source freshness
        env:
          DBT_USER: ${{ secrets.DBT_PROD_USER }}
          DBT_PASSWORD: ${{ secrets.DBT_PROD_PASSWORD }}
          DBT_HOST: ${{ secrets.DBT_PROD_HOST }}

      - name: dbt build (full)
        run: dbt build --target prod
        env:
          DBT_USER: ${{ secrets.DBT_PROD_USER }}
          DBT_PASSWORD: ${{ secrets.DBT_PROD_PASSWORD }}
          DBT_HOST: ${{ secrets.DBT_PROD_HOST }}

      - name: Upload manifest to S3 (new production baseline)
        run: |
          aws s3 cp target/manifest.json \
            s3://shopstream-dbt-artifacts/manifest.json
```

```
      env:
        AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
        AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}

      - name: Generate and publish docs
        run: |
          dbt docs generate --target prod
          aws s3 sync target/ s3://shopstream-dbt-docs/ --delete
```

State Selectors: Beyond state:modified

Selector	Selects
`state:modified`	Models whose SQL or config changed
`state:modified.body`	Models whose SQL body changed (not just config)
`state:modified.config`	Models whose config changed (but not SQL)
`state:new`	Models that didn't exist in the comparison state
`state:modified+`	Modified models plus all their downstream dependents
`1+state:modified+1`	Modified models plus 1 level up and 1 level down

Deferral: The Key to Fast CI

Deferral is what makes Slim CI *practical*. Without it, running `state:modified+` would fail for downstream models because their upstream dependencies weren't built in the CI environment. With `--defer`, dbt uses production versions of un-rebuilt models:

```
Example: stg_customers was changed on the PR branch.
```

```
Without --defer:
  dbt run --select state:modified+  →  builds stg_customers
                                       tries to build int_customer_order_histor
                                       (stg_orders doesn't exist in CI schema)

With --defer:
  dbt run --select state:modified+ --defer --state ./prod_manifest.json
  →  builds stg_customers (modified — uses CI schema)
  →  int_customer_order_history reads stg_orders from PRODUCTION (deferred)
  →  dim_customers reads int_customer_order_history from CI (just built)
  →  fct_orders reads dim_customers from CI (just built)
  ✅  Full downstream chain tested without building the entire project
```

The artifacts/ Pattern for Manifest Storage

bash

```bash
# Local development: compare against the last time you ran prod
# (useful for checking what your changes will affect)
dbt ls --select state:modified+ --state ~/.dbt/prod_manifest.json
```

Exercise 16 — Slim CI

Complete `exercises/ch12/exercise.md` (CI/CD exercises). The critical one: simulate a PR workflow locally. Run `dbt build`, copy the manifest, change one staging model, then run Slim CI pointing at the saved manifest. Observe which models ran vs. which were deferred.

CHAPTER 17 · PRODUCTION

Orchestration

Schedule and monitor your dbt pipelines in production with Airflow, Dagster, and Prefect.

The Orchestration Gap

dbt is not a scheduler. It has no concept of running on a cron, retrying on failure, alerting when something goes wrong, or coordinating with upstream data loaders. That's the orchestrator's job. dbt's CLI is the *worker*; Airflow/Dagster/Prefect is the *manager*.

Airflow Integration

The standard approach is to use the `DbtCloudRunJobOperator` (dbt Cloud) or the `DbtTaskGroup` from Astronomer Cosmos (dbt Core). Cosmos is the recommended approach for dbt Core + Airflow because it parses the dbt DAG and creates one Airflow task per dbt model, giving you fine-grained retry and visibility.

Option A: Simple Shell Operator (basic)

`dags/shopstream_dbt.py`

```python
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta

default_args = {
    "owner": "data-engineering",
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
    "email_on_failure": True,
    "email": ["data-alerts@shopstream.com"],
}

with DAG(
    dag_id="shopstream_dbt_daily",
    default_args=default_args,
    schedule_interval="0 6 * * *",   # 6am UTC daily
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=["dbt", "shopstream"],
) as dag:

    source_freshness = BashOperator(
        task_id="check_source_freshness",
        bash_command="cd /opt/dbt/shopstream && dbt source freshnes
    )

    dbt_snapshot = BashOperator(
        task_id="dbt_snapshot",
        bash_command="cd /opt/dbt/shopstream && dbt snapshot",
    )

    dbt_run = BashOperator(
        task_id="dbt_run",
        bash_command="cd /opt/dbt/shopstream && dbt run --target pr
    )

    dbt_test = BashOperator(
        task_id="dbt_test",
        bash_command="cd /opt/dbt/shopstream && dbt test --target p
    )
```

```
source_freshness >> dbt_snapshot >> dbt_run >> dbt_test
```

Option B: Astronomer Cosmos (recommended)

Cosmos parses `manifest.json` and creates one Airflow task per dbt node, giving you per-model retry, detailed lineage, and task-level SLAs:

`requirements.txt`

```
astronomer-cosmos[dbt-postgres]>=1.4.0
```

`dags/shopstream_dbt_cosmos.py`

```
from airflow import DAG
from airflow.operators.empty import EmptyOperator
from cosmos import DbtTaskGroup, ProjectConfig, ProfileConfig, Execu
from cosmos.profiles import PostgresUserPasswordProfileMapping
from datetime import datetime

profile_config = ProfileConfig(
    profile_name="shopstream",
    target_name="prod",
    profile_mapping=PostgresUserPasswordProfileMapping(
        conn_id="shopstream_postgres",
        profile_args={"schema": "analytics"},
    ),
)

project_config = ProjectConfig(
    dbt_project_path="/opt/dbt/shopstream",
)

with DAG(
    dag_id="shopstream_dbt_cosmos",
    schedule_interval="0 6 * * *",
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=["dbt", "cosmos"],
) as dag:

    pre_dbt = EmptyOperator(task_id="pre_dbt_checks")

    dbt_models = DbtTaskGroup(
        group_id="dbt_models",
        project_config=project_config,
        profile_config=profile_config,
        execution_config=ExecutionConfig(
            dbt_executable_path="/opt/dbt-env/bin/dbt",
        ),
        operator_args={
            "vars": {"min_date": "{{ ds }}"},  # pass Airflow templ
        },
    )
```

```
    post_dbt = EmptyOperator(task_id="notify_success")

    pre_dbt >> dbt_models >> post_dbt
```

Dagster Integration

Dagster has the deepest native dbt integration of any orchestrator. It reads the dbt manifest and creates Dagster assets (one per dbt model), with full lineage, partitioning, and data freshness monitoring built in:

`shopstream_dagster/assets.py`

```
from dagster import Definitions, define_asset_job, ScheduleDefiniti
from dagster_dbt import DbtCliResource, dbt_assets, DbtProject

dbt_project = DbtProject(project_dir="/opt/dbt/shopstream")
dbt_project.prepare_if_dev()

@dbt_assets(manifest=dbt_project.manifest_path)
def shopstream_dbt_assets(context, dbt: DbtCliResource):
    yield from dbt.cli(["build"], context=context).stream()

dbt_resource = DbtCliResource(project_dir=dbt_project)

shopstream_job = define_asset_job(
    name="shopstream_daily",
    selection=[shopstream_dbt_assets],
)

daily_schedule = ScheduleDefinition(
    job=shopstream_job,
    cron_schedule="0 6 * * *",
)

defs = Definitions(
    assets=[shopstream_dbt_assets],
    resources={"dbt": dbt_resource},
    jobs=[shopstream_job],
    schedules=[daily_schedule],
)
```

Orchestration Best Practices

Practice	Why
Run `dbt source freshness` before `dbt run`	Fail fast if upstream data is stale — don't waste compute building on bad data

Practice	Why
Run `dbt snapshot` before `dbt run`	Capture row-level history before transformation overwrites staging
Separate test DAG from run DAG	Allows test failures to alert without blocking new data landing
Use `dbt build` in CI, `dbt run` + `dbt test` separately in prod	Prod: you want data available ASAP even if some tests are still running
Pass `--vars '{"execution_date": "{{ ds }}"}'`	Makes dbt aware of which date Airflow is processing for backfill safety

Handling Failures and Retries

bash

```
# Resume a failed run from where it left off (skip already-successfu
dbt run --select result:error+ --state ./target

# Run only errored models and their descendants from the last run
dbt retry
```

CHAPTER 18 · PRODUCTION

Performance Optimization

Make your pipelines faster, cheaper, and more reliable — without rewriting your models.

Understand Before You Optimize

Before touching your models, understand where time is actually spent. Run `dbt run` with verbose logging and check your warehouse's query history:

bash

```
dbt run --debug 2>&1 | grep "Completed in"
# Model stg_customers: Completed in 0.3s
# Model int_orders_with_payments: Completed in 2.1s
# Model fct_orders: Completed in 45.2s    ← investigate this one
# Model fct_revenue_daily: Completed in 0.8s (incremental — already
```

Partitioning and Clustering

The single biggest performance lever for large tables. Add partitioning to tables that are always filtered by a date range:

BigQuery Partitioning

sql

```sql
{{
  config(
    materialized  = 'table',
    partition_by  = {
      "field": "order_date",
      "data_type": "date",
      "granularity": "month"   -- partition by month (day for high-
    },
    cluster_by    = ["customer_id", "status"]  -- sort within parti
  )
}}
```

Snowflake Clustering

sql

```sql
{{
  config(
    materialized   = 'table',
    cluster_by     = ['order_date', 'status']
    -- Snowflake auto-clustering handles partitioning automatically
  )
}}
```

Postgres Partitioning

sql

```
{{
  config(
    materialized         = 'table',
    partition_by         = 'order_date',
    partition_type       = 'range',
    partition_start      = "'2022-01-01'",
    partition_stop       = "'2025-12-31'",
    partition_every      = '1 month'
  )
}}
```

Incremental Models Are Your Best Friend

The fastest optimization is converting a slow full-refresh table to incremental. A model that takes 45 minutes to rebuild can become a 2-minute incremental run. Review the Chapter 10 patterns and apply them to your slowest models.

Identifying Good Incremental Candidates

A model is a good incremental candidate if:

- It has a timestamp column that reliably indicates when a row was created or updated
- Historical rows are never (or rarely) updated once written
- The table is large enough that rebuilding it is slow (>5 minutes)

Threads: Parallel Execution

dbt runs models in parallel up to the `threads` limit in your profile. Increasing threads is the easiest performance win for projects with many independent models:

`profiles.yml`

```
dev:
  threads: 4      # fine for development
prod:
  threads: 16     # use more in production — watch warehouse concurr
```

> **THREADS VS WAREHOUSE CONCURRENCY**
>
> Setting threads higher than your warehouse's concurrency limit will cause queries to queue rather than run in parallel. For Snowflake, your warehouse size determines concurrency. For BigQuery, the limit is project-level. Monitor your warehouse's query queue and increase threads incrementally.

Model Selection for Incremental Development

During active development, never run your entire project when you only need one model and its ancestors:

bash

```
# Build only fct_orders and everything upstream
dbt run --select +fct_orders

# Build only the models you changed today
dbt run --select state:modified+

# Build and test one model at a time
dbt build --select fct_orders
```

The Defer Pattern for Dev

In development, use `--defer` to avoid rebuilding unchanged upstream models. This can turn a 30-minute dev cycle into 2 minutes:

bash

```
# Download the prod manifest
aws s3 cp s3://shopstream-dbt-artifacts/manifest.json ~/.dbt/prod_ma

# Now run only the model you're working on, reading upstream deps fr
dbt run \
  --select fct_orders \
  --defer \
  --state ~/.dbt/prod_manifest.json
```

persist_docs: Sync dbt Docs to the Warehouse

Push your dbt column descriptions directly to the warehouse as table/column comments — so analysts see your documentation even when using a SQL client directly:

dbt_project.yml

```
models:
  shopstream:
    +persist_docs:
      relation: true    # sync model description as table comment
      columns: true     # sync column descriptions as column commen
```

Query Optimization Checklist

Issue	Fix
Full table scans on large fact tables	Add partitioning on the most common filter column (usually a date)
Slow JOINs on dimension tables	Add clustering/sort keys on join columns
Full rebuild of large tables every run	Convert to incremental
Models running sequentially that could be parallel	Increase threads; check for unnecessary ref() dependencies
Expensive subqueries repeated across multiple models	Materialize as a table (not ephemeral/view) so it's computed once
Slow staging views over large raw tables	Add a filter on `created_at` where possible, or materialize as incremental table
Unused columns in mart tables	Remove — fewer columns = smaller scans in columnar stores

Monitoring Build Performance Over Time

Track model build times by parsing the dbt artifacts after each run. The `run_results.json` file contains timing data for every node:

Python script for build time tracking

```
import json
from datetime import datetime

with open("target/run_results.json") as f:
    results = json.load(f)

# Print models sorted by execution time
nodes = [
    {
        "model": r["unique_id"].split(".")[-1],
        "status": r["status"],
        "seconds": round(r["execution_time"], 2),
    }
    for r in results["results"]
    if r.get("execution_time")
]

for node in sorted(nodes, key=lambda x: x["seconds"], reverse=True)
    print(f"{node['model']:50s}  {node['seconds']:6.1f}s  [{node['s
```

APPENDIX

Quick Reference

Essential commands, config options, and patterns at a glance.

CLI Commands Reference

Command	Common flags	Purpose
`dbt build`	`--select` `--exclude` `--full-refresh`	Run + test in DAG order
`dbt run`	`--select` `--full-refresh` `--threads`	Build models only
`dbt test`	`--select` `--store-failures`	Run tests only
`dbt seed`	`--select` `--full-refresh`	Load CSVs
`dbt snapshot`	`--select`	Run SCD Type 2
`dbt source freshness`	`--select`	Check source staleness
`dbt compile`	`--select`	Render Jinja without running

Command	Common flags	Purpose
`dbt ls`	`--select` `--output`	List matching nodes
`dbt debug`		Test warehouse connection
`dbt deps`		Install packages
`dbt clean`		Delete target/ and dbt_packages/
`dbt retry`		Re-run errored models from last run
`dbt run-operation`	`--args`	Execute a macro directly
`dbt docs generate`		Build docs artifacts
`dbt docs serve`	`--port`	Serve docs locally

Node Selection Syntax Reference

Syntax	Meaning
`model_name`	Exactly this model
`+model_name`	model_name and all its ancestors
`model_name+`	model_name and all its descendants
`+model_name+`	model_name, all ancestors, all descendants
`1+model_name`	model_name plus 1 level of ancestors

Syntax	Meaning
`path/to/dir`	All models in that directory
`tag:tagname`	All models with that tag
`source:source_name`	All models from a source
`exposure:exp_name`	An exposure node
`state:modified`	Models changed since last state
`state:new`	Models not in the comparison state
`result:error`	Models that errored in the last run
`A,B`	Intersection: in both A and B
`A B`	Union: in A or B

Config Options Reference

Option	Values	Applies to
`materialized`	view, table, incremental, ephemeral	All models
`schema`	string	All models
`alias`	string	All models
`tags`	list of strings	All models
`enabled`	true/false	All nodes
`unique_key`	string or list	Incremental

Option	Values	Applies to
`incremental_strategy`	append, merge, delete+insert, insert_overwrite, microbatch	Incremental
`on_schema_change`	ignore, fail, append_new_columns, sync_all_columns	Incremental
`incremental_predicates`	list of SQL strings	Incremental
`pre_hook` / `post_hook`	SQL string or list	All models
`grants`	dict (privilege: [roles])	All models
`persist_docs`	{relation: bool, columns: bool}	All models
`contract.enforced`	true/false	All models
`store_failures`	true/false	Tests

Jinja Functions Reference

Function	Purpose
`ref('model')`	Reference another model — creates DAG edge
`source('source', 'table')`	Reference a source table
`config(key=value)`	Set model configuration
`var('name', default)`	Access a project variable
`env_var('ENV_VAR', default)`	Access an environment variable
`is_incremental()`	True during incremental (non-full-refresh) runs

Function	Purpose
`this`	The current model's relation (database object)
`target`	Object with name, schema, database, type, threads
`adapter.type()`	The warehouse adapter type (postgres, bigquery, etc.)
`run_query(sql)`	Execute SQL at compile time and return results
`log(msg, info=True)`	Print a message during compilation/execution
`execute`	True during execution pass, False during parse pass
`modules.datetime`	Python datetime module (accessible in Jinja)